

Functional Programming

Lambda expressions; functional interfaces (Predicate, Function, Consumer, Supplier); method references

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Introduction to Functional Programming

Functional programming is a programming paradigm in which functions are used to express computations and transform data. Java introduced lambda expressions, method references, and the `java.util.function` package in Java 8, making it easier to write code in a functional style.

Important characteristics: Functions can be passed as arguments to methods.

- Lambda expressions reduce boilerplate code.
- Functional interfaces provide a target type for lambda expressions.
- Method references allow existing methods to be reused concisely.
- Functional-style code often emphasizes immutability and avoiding side effects.

Object-Oriented vs Functional Style

Object-Oriented vs Functional Style

Feature	Object-Oriented Programming	Functional Programming
Main focus	Objects and classes	Functions and transformations
State	Often mutable	Immutability preferred
Code organization	Classes and methods	Functions and function composition
Reusability	Inheritance, interfaces, composition	Higher-order functions, composition
Java support	Core Java	Lambdas, functional interfaces, streams
Example	Class implementing an interface	Lambda passed to a method



Java Lambda Expressions

lambda expressions, introduced in Java 8, allow developers to write concise, functional-style code by representing anonymous functions. They enable passing code as parameters or assigning it to variables, resulting in cleaner and more readable programs.

- It implement a functional interface (An interface with only one abstract function)
- Enable passing code as data (method arguments).
- Lambda expressions can access only final or effectively final variables from the enclosing scope.

Syntax

- The simplest lambda expression contains a single parameter and an expression:
- **parameter -> expression**

Java Lambda Expressions

To use more than one parameter, wrap them in parentheses:

`(parameter1, parameter2) -> expression`

Simple expressions must return a value immediately. They cannot contain multiple statements, such as loops or if conditions. To do more complex work, use a code block with curly braces. If the lambda should return a value, use the return keyword:

`(parameter1, parameter2) -> {`

`// code block`

`return result;`

`}`

AI Integration in Java

1. Predicate<T>

Purpose : Predicate is used when you want to test/check something.

- Takes one input
- Returns Boolean
- Main method: test()

2. Function<T, R>

Purpose: Function is used when you want to take an input and produce a result.

- Takes one input
- Returns one output
- Main method: apply()

3. Consumer<T>

Purpose: Consumer is used when you want to perform an operation on a value.

- Takes one input
- Returns nothing
- Main method: accept()

4. Supplier<T>

Purpose: Supplier is used when you want to supply/generate a value.

- Takes no input
- Returns one value
- Main method: get()

All Four at a Glance

Interface	Input	Output	Method	Common Use
Predicate<T>	1	boolean	test()	Check/Test
Function<T,R>	1	1	apply()	Transform
Consumer<T>	1	Nothing	accept()	Perform action
Supplier<T>	Nothing	1	get()	Supply value

Method References

- A method reference is shorthand for a compatible lambda that invokes an existing method. Four forms: `ClassName::staticMethod`; `object::instanceMethod`; `ClassName::instanceMethod` (receiver is the first lambda argument); `ClassName::new` (constructor reference). A method reference still requires a compatible functional-interface target.

Kind	Lambda	Reference
Static method	<code>s -> Integer.parseInt(s)</code>	<code>Integer::parseInt</code>
Bound instance method	<code>() -> message.toUpperCase()</code>	<code>message::toUpperCase</code>
Unbound instance method	<code>s -> s.length()</code>	<code>String::length</code>
Constructor	<code>() -> new ArrayList<String>()</code>	<code>ArrayList<String>::new</code>

Working with Collections and Streams

- Lambdas are frequently used with `removeIf`, `forEach`, `sort` and the Stream API. `filter` expects a Predicate; `map` expects a Function; `forEach` expects a Consumer; `orElseGet` accepts a Supplier.
- `import java.util.*;`
- `import java.util.stream.Collectors;`
- `public class StreamDemo {`
- `public static void main(String[] args) {`
- `List<String> names = Arrays.asList("Asha", "Raj", "Anita", "Ram");`
- `List<String> result = names.stream()`
- `.filter(s -> s.length() > 3)`
- `.map(String::toUpperCase)`
- `.sorted()`
- `.collect(Collectors.toList());`
- `result.forEach(System.out::println);`
- `String missing =`
 `Optional.<String>empty()`
- `.orElseGet(() -> "No name");`
- `System.out.println(missing);`
- `}`
- `}`
- `// Output: ANITA`
- `// ASHA`
- `// No name`

Comparisons

Criteria	Lambda expression	Method reference	
Purpose	Defines behavior inline	Reuses existing method/constructor	
Syntax	<code>x -> x.trim()</code>	<code>String::trim</code>	
When preferred	Custom logic or multiple operations	Direct forwarding to named method	
Target	Functional interface	Functional interface	
Interface	Returns value?	Primary question	Example
Predicate	boolean	Does it satisfy condition?	<code>n -> n > 18</code>
Function	Yes (R)	What is transformed value?	<code>s -> s.length()</code>
Consumer	No (void)	What action to perform?	<code>System.out::println</code>
Supplier	Yes (T)	What value to provide?	<code>() -> 100</code>